

3 Evaluation Protocol

Here we outline the evaluation protocol we use to answer the research question “To what extent can LLMs resolve conversational implicature?”. We focus on binary implicatures that imply “yes” or “no” (see Figure 1). We say a model resolves an implicature correctly if it assigns higher likelihood to a coherent utterance than a similar but incoherent one, detailed below.

Zero-shot evaluation. Consider the example from the introduction packed into a single utterance:

Esther asked “Can you come to my party on Friday?” and Juan responded “I have to work”, which means no.

We can transform this example to be *pragmatically incoherent* (in the sense that it will become pragmatically inconsistent with expected use) by replacing the word “no” with “yes”:

Esther asked “Can you come to my party on Friday?” and Juan responded “I have to work”, which means yes.

To resolve the implicature, the model should assign higher likelihood to the first of the two sentences above, namely the most coherent one. Importantly, both sentences have exactly the same words except for the binary implicature “yes” or “no”, making the assigned likelihood scores directly comparable. Formally, let the coherent prompt be $\mathbf{x}$ and the augmented, incoherent prompt be $\hat{\mathbf{x}}$. A model outputs a likelihood p parameterized by weights θ . We say a model correctly resolves an example $\mathbf{x}$ when it assigns $p_\theta(\mathbf{x}) > p_\theta(\hat{\mathbf{x}})$. This is equivalent to evaluating whether the model assigns a higher likelihood to the correct continuation of the two options. Note that this is a more lenient evaluation protocol than sometimes used for language models, where models are evaluated on their ability to generate the correct continuation, in this case “no”. The greedy decoding approach (evaluating whether “yes” or “no” is generated) is also captured by our approach, but we additionally label an example correct if “no” is not the highest assigned likelihood, but still higher than “yes”. We did not opt for greedy decoding because “no” is not the only coherent continuation here, and marginalising over all possible correct continuations is intractable. The more lenient evaluation does capture implicature resolution, because the choice of “no” versus “yes” is only determined by the resolution of the implicature. We guide the models to output “yes” or “no” explicitly in three of the six prompt templates with instructions, such that we can estimate the effect of this guidance on performance. For two model classes (i.e. GPT-3.5-turbo and GPT-4) we do not have access to likelihoods, and for these models we take the greedy decoding approach, guiding the model to output “yes” or “no” explicitly in all prompts (see Table 6 in Appendix F).

We use a dataset of conversational implicatures curated by George and Mamidi (2020)⁵. It contains implicatures that, like in Figure 1, are presented in utterance-response-implicature tuples. Of these, 718 are binary implicatures that we can convert into an incoherent sentence. We randomly sample 600 examples for the test set and keep the remaining 118 as a development set to improve implicature resolution after pre-training through in-context prompting or fine-tuning.

Few-shot in-context evaluation. We add k examples of the task to the prompt, e.g. with $k = 2$:

Esther asked “Have you found him yet?” and Juan responded “They’re still looking”, which means no.

Esther asked “Are you having fun?” and Juan responded “Is the pope Catholic?”, which means yes.

Finish the following sentence:

Esther asked “Can you come to my party on Friday?” and Juan responded “I have to work”, which means no.

We evaluate the models’ k -shot capabilities for $k \in \{1, 5, 10, 15, 30\}$ by randomly sampling k examples from the development set for each test example. We opt for a random sampling approach to control for two sources of randomness. Firstly, examples have different levels of informativeness. Secondly, recent work found that the order in which examples are presented matters (Lu et al., 2022). Ideally, to marginalise over these random factors, we would evaluate each test example with all

⁵Published under a CC BY 4.0 license.

permutations of k examples from the development set. This requires $\frac{118!}{(118-k)!}$ evaluations for each test example, which is intractable. Instead, we estimate performance per test example by randomly sampling from the development set. In this way we control for some of the variance in performance, but avoid extra evaluations. We ensure each model sees the same few-shot examples per test example.

Controlling for prompt sensitivity. It has been shown language models are sensitive to prompt wording (Efrat and Levy, 2020; Tan et al., 2021; Reynolds and McDonell, 2021a; Webson and Pavlick, 2021). To control for this factor of randomness we manually curate six different template prompts and measure performance across these. One of the templates has been presented above, namely “Esther asked *<utterance>* and Juan responded *<response>*, which means *<implicature>*”. Another template is: “Question: *<utterance>*, response: *<response>*, meaning: *<implicature>*”. The former we call *natural* prompts and the latter *structured* prompts. Each group has three templates that only differ slightly in wording. This grouping allows us to look at the variance due to slight changes in wording as well as performance difference due to a completely different way of presenting the example. The full list of prompts can be found in Appendix F.

4 Experiments

The set of large language model classes we evaluate can be grouped into four distinct categories:

1. *Base models*: large-scale pre-trained models; RoBERTa (Liu et al., 2019), BERT (Devlin et al., 2018), GPT-2 (Radford et al., 2019), EleutherAI (Wang and Komatsuzaki, 2021; Black et al., 2022), BLOOM (BigScience, 2022), OPT (Zhang et al., 2022), Cohere’s base models, and GPT-3 (Brown et al., 2020)
2. *Dialogue FT*: LLMs fine-tuned on dialogue, BlenderBot (Ng et al., 2019).
3. *Benchmark IT*: LLMs fine-tuned on tasks with natural instructions for each benchmark or “benchmark-level instruction-tuned models”; T0 (Sanh et al., 2022) and Flan-T5 (Chung et al., 2022).
4. *Example IT*: LLMs fine-tuned on tasks with natural instructions for each example or “example-level instruction-tuned models”; a subset of OpenAI’s API models and Cohere’s API models).

For Benchmark IT models, annotators write a single instruction for an entire dataset. The models are then fine-tuned on each example from the dataset with the same instruction. We distinguish this from example-level IT; for that type of fine-tuning each example in a dataset gets a new instruction, resulting in a more diverse dataset. Each group contains model classes for which we evaluate a range of sizes. A detailed categorization of the models and their attributes can be found in appendix G.⁶ We make use of the OpenAI and Cohere APIs as well as the pretrained models in the transformers library (Wolf et al., 2020) and EleutherAI’s framework to evaluate them (Gao et al., 2021). All code used for this paper can be found on GitHub⁷ and the dataset is made publicly available on HuggingFace⁸. Below, we present zero-shot and few-shot results, discussing patterns of performance of the models in the four different groups. We further look at the results for different model sizes of each model class and the variance over the prompt templates. We contrast the models’ performance with human performance. To this end, each test example gets annotated by five humans. We split the test set in four and assign each annotator a subset, leaving us with twenty annotators in total. The average human performance is 86.2%, and the best performance is 92%. Some of the errors humans make uncover examples that have multiple interpretations, and others uncover annotation errors. The nature of the task of implicature resolution means we do not expect models to perform better than human best performance. Details on the human experiment can be found in the Appendix H (also containing an analysis of human errors), and detailed results per model and prompt template in Appendix K.¹⁰ We also test for spurious correlations present in the benchmark (like lexical cues the model can rely on), and find no indication (Appendix K.8).

Insight 1: Models instruction-tuned at the example level outperform all others. Table 1 shows the best 0-, 1-, and 5-shot accuracy each model class achieved on the implicature task. The best overall

⁶Note that there are several important aspects unknown for models behind APIs, like OpenAI’s model sizes.

⁷<https://github.com/LauraRuis/do-pigs-fly>

⁸<https://huggingface.co/datasets/UCL-DARK/ludwig>

Table 1: The k -shot accuracy ($k \in \{0, 1, 5\}$) for the best performing model of each class. For each model, we select the model size to show by choosing the one that achieves the best 5-shot performance. The std is over prompt templates for the models and over annotators for humans. FT stands for fine-tuning and IT for instruction-tuning. We find that the models in the *Example IT* class obtain significantly higher performance than all others. $\star$ means size unknown.

	Model	0-shot	1-shot	5-shot
Baselines and Toplines	Random		50%	
	Human avg.		86.2% $\pm$ 2.3	
Base models	BERT-110M	54.8% $\pm$ 1.6	51.7% $\pm$ 1.7	53.3% $\pm$ 2.2
	RoBERTa-355M	55.6% $\pm$ 2.0	54.1% $\pm$ 0.9	57.1% $\pm$ 1.5
	GPT-2-xl	51.3% $\pm$ 2.9	57.4% $\pm$ 3.3	57.7% $\pm$ 1.1
	EleutherAI-20B	57.5% $\pm$ 3.3	55.9% $\pm$ 2.3	61.1% $\pm$ 4.9
	BLOOM-176B	54.2% $\pm$ 1.2	61.1% $\pm$ 2.7	65.4% $\pm$ 3.4
	OPT-13B	61.0% $\pm$ 5.5	60.6% $\pm$ 2.7	67.4% $\pm$ 2.1
	Cohere-52B	58.5% $\pm$ 4.0	63.0% $\pm$ 3.8	65.1% $\pm$ 2.9
	GPT-3-175B	57.7% $\pm$ 4.4	65.7% $\pm$ 1.4	68.7% $\pm$ 1.5
Dialogue FT	BlenderBot-2.7B	53.4% $\pm$ 0.3	53.3% $\pm$ 0.1	53.3% $\pm$ 0.1
Benchmark IT	T0-11B	55.6% $\pm$ 7.0	47.8% $\pm$ 0.5	47.0% $\pm$ 0.2
	Flan-T5-11B	60.8% $\pm$ 2.4	57.4% $\pm$ 5.0	61.7% $\pm$ 4.8
Example IT	text-davinci-001- $\star$	72.3% $\pm$ 2.8	72.7% $\pm$ 1.3	74.5% $\pm$ 1.0
	text-davinci-002- $\star$	70.6% $\pm$ 2.3	75.6% $\pm$ 2.8	79.6% $\pm$ 2.0
	text-davinci-003- $\star$	71.2% $\pm$ 2.8	74.3% $\pm$ 1.4	79.7% $\pm$ 0.6
	ChatGPT- $\star$	72.1% $\pm$ 5.9	75.1% $\pm$ 1.5	73.9% $\pm$ 6.3
	GPT-4- $\star$	81.8% $\pm$ 1.8	82.3% $\pm$ 1.4	82.0% $\pm$ 1.7
	Cohere-command-52B	60.2% $\pm$ 5.2	72.8% $\pm$ 1.3	75.4% $\pm$ 1.8

accuracy is achieved by GPT-4 (the size of this model is unknown) at $82.3\% \pm 1.4$. This leaves a gap of 3.9% with human average performance. All models in the class *Example IT* perform significantly better than any of the other models for all k , except Cohere-command-52b at 0-shot. This result is more clearly seen in Figure 2, where we present the average accuracy for each model group. The performance for the other model classes across k ranges from 47.0% by BlenderBot-2.7b at $k = 5$ and 68.7% by GPT-3-175b at $k = 5$. Even though base models benefit from few-shot examples, their performance remains mostly closer to random than to humans for all k , showing a gap of at least 17.5% with the average human. We observe a decrease in performance for $k > 0$ for the group *Benchmark IT*. This is not surprising, as these kind of models are specifically fine-tuned to be better at zero-shot generalisation (Sanh et al., 2022; Chung et al., 2022). BlenderBot, in the group *Dialogue FT*, performs barely better than random for all k . We hypothesise that the lower performance which Cohere-command-52b achieves 0-shot is not due to a lack of implicature understanding, but due to a failure to calibrate the yes/no likelihoods without examples. For this model, we observe a sharp rise in performance from $k = 0$ to $k = 1$ (see Table 1 or Figure 2). Since it is unlikely that one example of an implicature induces pragmatic understanding, we hypothesise that few-shot prompting mostly serves to clarify the task format. We test this hypothesis in Appendix K.6 by repeating the 1- and 5-shot experiment with random labels for Cohere-command-52B and text-davinci-001. We find that the performance does not degrade, which confirms that the few-shot examples mainly serve to prime the model towards producing outputs following the yes/no structure.

Insight 2: The results are robust to different prompt templates. As detailed in Section 3, each example in the test set is wrapped in six different prompt templates. The standard deviation in Table 1 and in Figure 2 shows the sensitivity to different prompt wording. The standard deviation ranges from 0.3 for BlenderBot to 7.0 for T0-11B. All in all, the sensitivity to prompt wording does not seem to be a problem for this task; when taking into account the confidence intervals the result remains that models in the group *Example IT* perform significantly better than all other models, but worse than humans. In Appendix K.4 another analysis is presented that shows how different prompt templates benefit from in-context examples. The takeaway from the analysis is that in-context prompting can mitigate the fact that some models are better at natural prompts and others better at structured prompts by improving performance on the type of prompt the model struggles with zero-shot. Again, when